



Introducción a LangGraph

Agentes como grafos de estado · Panorámica general

MUIA — Máster Universitario en Inteligencia Artificial

Escuela Superior de Informática · Universidad de Castilla-La Mancha

Panorámica

En una frase

LangGraph es una biblioteca para construir agentes como grafos de estado: tú declaras los nodos, las conexiones y qué información viaja entre ellos; la biblioteca ejecuta el ciclo, guarda el estado y te deja entrar a supervisarlo.

> *Si un agente es «un modelo de lenguaje que razona, actúa y **vuelve a decidir**», LangGraph es el formalismo que hace explícito ese volver a decidir.*

El problema que resuelve

Las cadenas clásicas son **acíclicas**: cada paso ocurre una vez y el control no vuelve atrás.

	Cadena (DAG)	Agente
Flujo	lineal, sin retorno	**con ciclos**
Quién decide	el programador	el modelo, en ejecución
Estado	pasa de paso en paso	evoluciona y se conserva
Depurar	leer el *pipeline*	inspeccionar la traza

Un agente necesita **volver a decidir con la observación en la mano**. Eso es un ciclo, y un ciclo no cabe en una cadena.

La idea que lo cambia todo

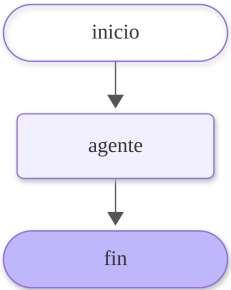
En una cadena, la estructura del agente está **dentro de la biblioteca**.

En LangGraph, la estructura es **un objeto que se puede imprimir**:

```
grafo.get_graph().draw_mermaid()
```

Eso convierte al agente en algo que se **lee, se revisa y se modifica**, en lugar de algo que solo se ejecuta.

El grafo más pequeño posible



{Tres piezas y ya hay un programa: un estado, un nodo y dos aristas.}

Las piezas

Cuatro piezas, ni una más

Pieza	Qué es	En el código
Estado	lo que viaja entre nodos	`TypedDict` con anotaciones
Nodos	unidades de cómputo	funciones que devuelven un parche
Aristas	el orden y las decisiones	`add_edge` · `add_conditional_edges`
Runtime	ejecuta, guarda y pausa	`compile()`

Todo lo demás que verás —memoria, supervisión, streaming, varios agentes— se construye **encima de estas cuatro piezas**, no al lado.

El estado es un sobre, no una pizarra

Un nodo **no devuelve el estado completo**: devuelve solo las claves que modifica.

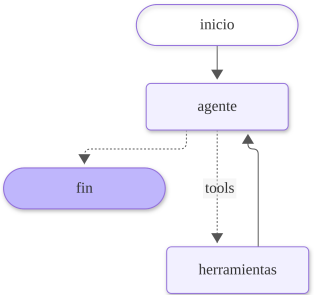
```python
def nodo(estado):
    return {"messages": [respuesta]}   # <- un parche, no el estado
```

La biblioteca fusiona ese parche con el estado anterior mediante un *reducer*.

Elegir el *reducer* es una decisión **semántica**: con `add_messages` el historial **crece**; sin él, cada nodo lo **sobrescribe** y el agente pierde la conversación **sin dar ningún error**.

Las aristas condicionales son el bucle

```
START -> agente -> ¿pidió herramientas? -> herramientas -> agente -> ...
```



{Ese retorno de «herramientas» a «agente» es el bucle ReAct, y está a la vista.}

Lo que te da

Cuatro capacidades que no tienes que programar

Capacidad	Qué te ahorra
Persistencia	memoria por conversación, sin gestión manual
Interrupciones	aprobar o corregir antes de actuar
Streaming	mostrar el avance mientras se produce
Multiagente	varios especialistas sobre el mismo formalismo

No son añadidos externos: son propiedades del **runtime** que ejecuta el grafo.

Memoria: un identificador y una clave

```
grafo = constructor.compile(checkpointer=InMemorySaver())

grafo.invoke(entrada, config={"configurable": {"thread_id": "alumno-A"}})
```

- Mismo `thread_id` → misma conversación. Otro identificador → conversación nueva, aislada.
- La memoria a **largo plazo** (hechos del usuario) vive aparte, en un almacén.
- Y el aviso que cuesta caro: **olvidar el `checkpointer` no da error**; simplemente no hay memoria.

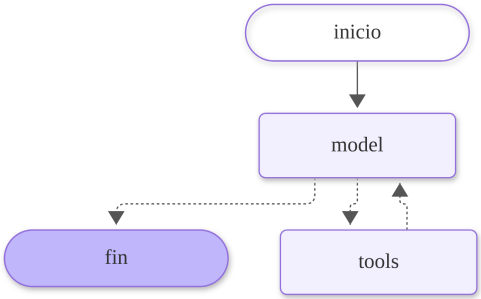
Interrupción: el freno de mano

```python
def nodo_compra(estado):
    decision = interrupt({"pregunta": "¿Aprobamos el gasto?"})
    return {"messages": [AIMessage(content=str(decision))]}
```

```python
parada = grafo.invoke(entrada, config=cfg)              # se detiene aquí
final  = grafo.invoke(Command(resume="sí"), config=cfg)  # y continúa
```

Necesita `checkpointer`: es lo que conserva el estado mientras el grafo está parado.

Por dentro, esto es lo que usa `create_agent`



{La función de conveniencia construye el mismo ciclo, con más casos límite cubiertos.}

Para terminar

Cuándo merece la pena

Sí:

- el agente necesita **ciclos**: razonar, actuar y volver a decidir;
- quieres **ver y auditar** la estructura, no solo ejecutarla;
- necesitas **memoria, supervisión humana o varios agentes**.

No:

- una llamada única de *prompt* a respuesta;
- un *pipeline* lineal sin decisiones;
- cuando la función de conveniencia ya te resuelve el caso.

Primer contacto

```python
from langgraph.graph import StateGraph, START, END, MessagesState

def nodo(estado: MessagesState):
    return {"messages": [modelo.invoke(estado["messages"])]}

c = StateGraph(MessagesState)
c.add_node("agente", nodo)
c.add_edge(START, "agente")
c.add_edge("agente", END)
grafo = c.compile()

grafo.invoke({"messages": [{"role": "user", "content": "Hola"}]})
```

Lo que conviene recordar

Un agente es un grafo.

El estado es el sobre.

Las aristas condicionales son las decisiones.

La memoria y el freno vienen de serie.

{Material completo: libretas ejecutables, guía docente, manual de instalación y texto académico.}